Problems on Trees

DAY 173
30/12/2025

Question 3 : Give an algorithm for Searching an Element In the Binary tree

Solution : Given a Binary Tree, return true if a node with data is found in the tree. Recurse down the tree, choose the left or Right Branch By comparing data with Each node's data.

Algorithm

1. if the tree is empty, return false
2. if Current node's data Equals the key, return true
3. Recursively Search the left Subtree
4. if found In the left subtree, return true
5. otherwise, recursively Search the right Subtree
6. Return the result

```
static boolean Search (TreeNode root, int key) {

    if (root == null)
        return false;

    if (root.data == key)
        return true;

    return Search (root.left, key) || Search (root.right, key);
}
```

Question 4 Give an algorithm for Searching an element in a Binary tree Without Recursion.

We can use level order traversal for solving this problem. The only change required in level order traversal is, instead of printing the data, We just need to check Weather root data is Equal to the element We Want to Search.

Algorithm

1. if the tree is Empty, Return false
2. Create an Empty Queue
3. Insert the Root node Into Queue
4. While the Queue is not Empty
   - Remove the node from the queue
   - If node's data Equals the key, return true
   - Insert left child into Queue (if Exists)
   - Insert right child into Queue (if Exists)
5. If traversal Ends and key is not found, return false.

```
static boolean Search (TreeNode root, int key) {
    if (root == null)
        return false;

    Queue<TreeNode> q = new LinkedList<>();
    q.add(root);

    While (!q.isEmpty()) {

        TreeNode cur = q.poll();
```

```
        if (cur.data == key)
            return true;

        if (cur.left != null)
            q.add(cur.left);

        if (cur.right != null)
            q.add(cur.right);
    }

    return false;
}
```